МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»

Дисциплина Операционные системы

# Лабораторная работа № 4
# УПРАВЛЕНИЕ ПОТОКАМИ

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: Ассистент Савельева М. Г.

Минск, 2026

**Содержание**

Введение....3
1 Теоретические сведения....4
1.1 Поток и процесс....4
1.2 Потоки в *Windows*....4
1.3 Потоки в *Linux*....4
1.4 *Thread Local Storage*....4
1.5 Волокна....5
1.6 Выводы по разделу....5
2 Выполнение заданий для *Windows*....6
2.1 Подготовка среды и сборка....6
2.2 Уровень 4–5: базовые операции с потоками....6
2.3 Уровень 7–8: простые числа и *TLS*....11
2.4 Уровень 9–10: волокна и число Пи....13
2.5 Выводы по разделу....14
3 Выполнение заданий для *Linux*....16
3.1 Подготовка среды и сборка....16
3.2 Уровень 4–5: базовые операции с потоками....16
3.3 Уровень 7–8: простые числа и *TLS*....17
3.4 Уровень 9–10: волокна и число Пи....19
3.5 Сравнение потоков и процессов....20
3.6 Выводы по разделу....20
Заключение....21
ПРИЛОЖЕНИЕ А....22

## Введение

Цель работы – получить практические навыки работы с потоками в операционных системах *Windows* и *Linux*.

Задачи:

– реализовать функцию *Lab*_04*x* с выводом идентификаторов процесса и потока, номера итерации и символов имени пользователя и ФИО;

– выполнить приложения *Lab*-04*a*, *Lab*-04*b*, *Lab*-04*c* и *Lab*-04*d* для *Windows*;

– реализовать поиск простых чисел в потоках с *TLS* через *WinAPI* и через __*declspec*(*thread*);

– реализовать вычисление числа Пи на волокнах с завершением по *Ctrl+C*;

– выполнить аналогичные приложения для *Linux* на *pthread* и *ucontext*;

– снять замеры производительности и сравнить потоки с процессами;

– сопоставить результаты и сделать выводы.

Работа выполнена в двух средах: *Windows* 11 *ARM* с компилятором *MSVC* и *Arch Linux ARM* с компилятором *GCC*. *Linux*-машина имеет 6 физических ядер, поэтому серия замеров включает 1, 2, 4, 8 и 12 потоков или процессов.

# 1 Теоретические сведения

## 1.1 Поток и процесс

Процесс – это выполняемая программа вместе с адресным пространством, открытыми файлами и другими ресурсами. Поток – единица выполнения внутри процесса. Потоки одного процесса разделяют адресное пространство и ресурсы, но имеют собственный стек, регистровый контекст и идентификатор.

Многопоточность позволяет выполнять несколько последовательностей инструкций в одном процессе. Переключение между потоками одного процесса обычно дешевле переключения между процессами, поскольку не требует смены адресного пространства. Однако общие изменяемые данные требуют синхронизации, иначе возникают гонки.

## 1.2 Потоки в *Windows*

Поток создаётся функцией *CreateThread*. Родитель получает дескриптор и идентификатор потока, а для ожидания завершения использует *WaitForSingleObject* или *WaitForMultipleObjects*. Приостановка и продолжение выполняются функциями *SuspendThread* и *ResumeThread*, принудительное завершение – *TerminateThread*.

Список потоков процесса можно получить через *Tool Help Library*: *CreateToolhelp*32*Snapshot* с флагом *TH*32*CS_SNAPTHREAD*, затем *Thread*32*First* и *Thread*32*Next*. Для каждого потока доступны идентификатор, приоритет и идентификатор процесса-владельца.

## 1.3 Потоки в *Linux*

В *POSIX*-системах поток создаётся функцией *pthread_create*, а родитель ожидает его завершения через *pthread_join*. Идентификатор потока, видимый ядру, возвращает системный вызов *gettid*; он же отображается как *LWP* в выводе *ps -L*.

Сведения о потоках процесса доступны через файловую систему */proc*: каталог */proc*/*PID*/*task* содержит подкаталоги с идентификаторами потоков, а в */proc*/*PID*/*status* есть строка *Threads*. Утилита *ps* с ключом *-L* показывает потоки и их состояния.

## 1.4 *Thread Local Storage*

*Thread Local Storage* (*TLS*) – механизм, при котором одна и та же переменная имеет отдельный экземпляр в каждом потоке. *TLS* применяется для буферов, счётчиков и других данных, которые не должны разделяться между потоками.

В *Windows TLS* через *WinAPI* строится на *TlsAlloc*, *TlsSetValue* и *TlsGetValue*. Компилятор *MSVC* поддерживает модификатор *__declspec*(*thread*). В *Linux* аналогом служат *pthread_key_create*, *pthread_setspecific* и *pthread_getspecific*; инициализация ключа выполняется через *pthread_once*. Модификатор *__thread* является аналогом *__declspec*(*thread*).

### 1.5 Волокна

Волокно – кооперативная единица выполнения, которая переключается только явно, без участия планировщика операционной системы. Волокна одного потока выполняются на одном ядре процессора и не могут выполняться параллельно.

В *Windows* волокна создаются функциями *ConvertThreadToFiber*, *CreateFiber* и *SwitchToFiber*. В *Linux* аналогичный механизм предоставляет семейство *ucontext*: *getcontext*, *makecontext*, *swapcontext* и *setcontext*. Волокна не видны ядру как отдельные потоки, поэтому *ps -L* показывает только один поток.

### 1.6 Выводы по разделу

Рассмотрены потоки и процессы, средства создания и ожидания потоков в *Windows* и *Linux*, механизм *TLS* и волокна. Показано, что потоки разделяют адресное пространство, а *TLS* даёт каждому потоку собственный экземпляр данных.

Волокна отличаются от потоков кооперативным переключением и невидимостью для планировщика. Эти теоретические положения используются в последующих разделах при разборе реализации.

## 2 Выполнение заданий для *Windows*

### 2.1 Подготовка среды и сборка

Исходные тексты *Windows*-части находятся в каталоге *lab*4/*windows*. Сборка выполняется командным файлом *build.cmd* с использованием *MSVC ARM*64, ключей /*utf*-8 и /*O*2. Результат сборки приведён на рисунке 2.1.

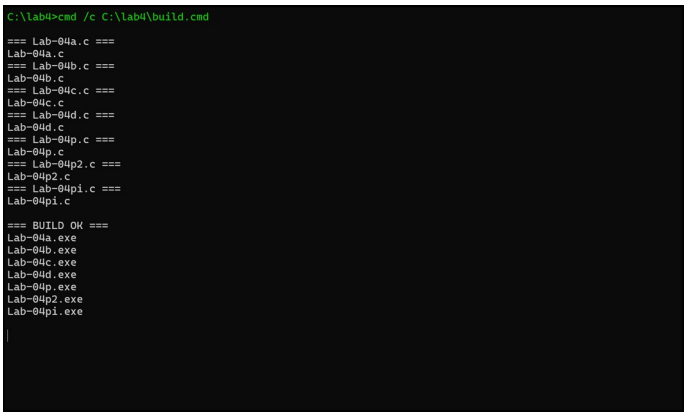



Рисунок 2.1 – Сборка *Windows*-приложений: сообщение *BUILD OK* и список из семи исполняемых файлов

Собраны семь приложений: *Lab*-04*a*, *Lab*-04*b*, *Lab*-04*c*, *Lab*-04*d*, *Lab*-04*p*, *Lab*-04*p*2 и *Lab*-04*pi*. Вывод кириллицы организован через заголовок *console_ru.h*, который перекодирует строки из *UTF*-8 в кодовую страницу консоли 866. Общие функции *Lab*_04*x* и перекодировка вывода вынесены в листинги А.14 и А.15.

### 2.2 Уровень 4–5: базовые операции с потоками

Функция *Lab*_04*x* выполняет заданное число итераций. На каждой итерации она печатает идентификатор процесса, идентификатор потока, номер итерации, очередной символ имени пользователя *User*-6*d*4*ec*851 и очередной символ ФИО Марушко Тимофей. Между итерациями выполняется задержка 400 мс.

В приложении *Lab*-04*a* создаются два дочерних потока на 55 и 130 итераций, а главный поток вызывает *Lab*_04*x* на 100 итераций. Главный поток ожидает завершения обоих дочерних потоков. Первые итерации трёх потоков показаны на рисунке 2.2, завершение – на рисунке 2.3. Исходный текст *Lab*-04*a* приведён в листинге А.7.

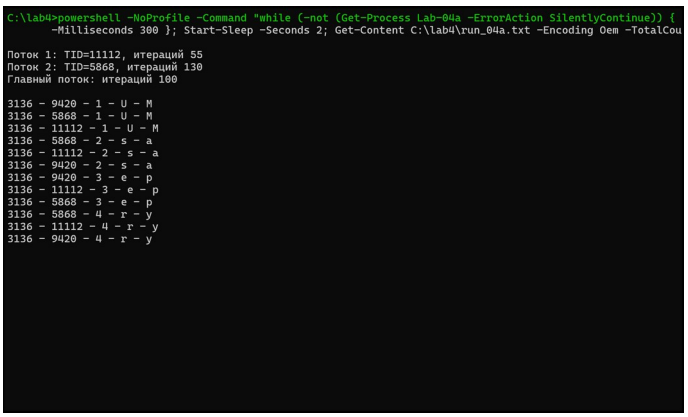



Рисунок 2.2 – *Lab*-04*a*: первые итерации главного и двух дочерних потоков

Завершение работы программы *Lab*-04*a* показано на рисунке 2.3.

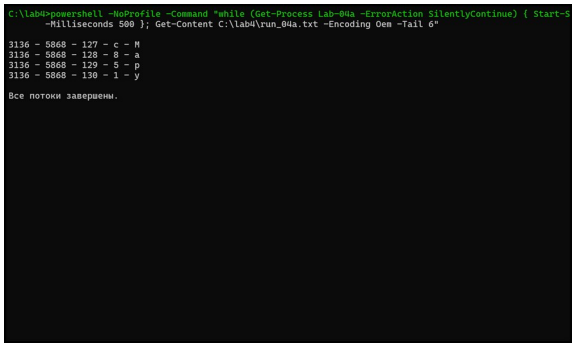



Рисунок 2.3 – *Lab*-04*a*: последние итерации и сообщение о завершении всех потоков

Список потоков работающего процесса *Lab*-04*a* получен через *PowerShell* командой *Get-Process* с выводом свойства *Threads*. На рисунке 2.4 видны три потока и их состояния ожидания.

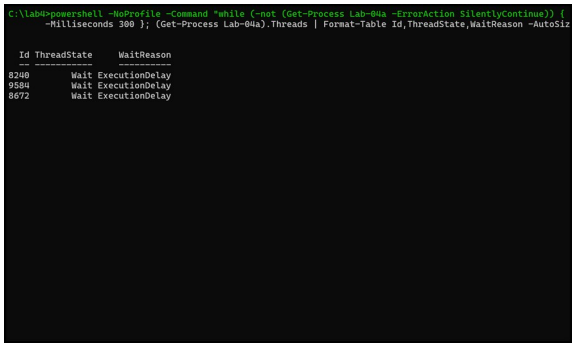



Рисунок 2.4 – *Lab*-04*a*: три потока процесса по данным *PowerShell*

Три потока.

Тот же процесс показан в *Process Explorer* с включённой нижней панелью *Threads*. Кадр на рисунке 2.5 подтверждает, что дочерние потоки являются потоками одного процесса.



Рисунок 2.5 – *Process Explorer*: потоки процесса *Lab*-04*a*

В приложении *Lab*-04*b* главный поток вместо вызова *Lab*_04*x* выполняет собственный цикл. На 30-й итерации он приостанавливает первый дочерний поток, на 70-й – возобновляет. Второй поток приостанавливается на 40-й итерации и возобновляется после завершения цикла главного потока. Последовательность событий приведена на рисунке 2.6. Исходный текст *Lab*-04*b* приведён в листинге А.8.

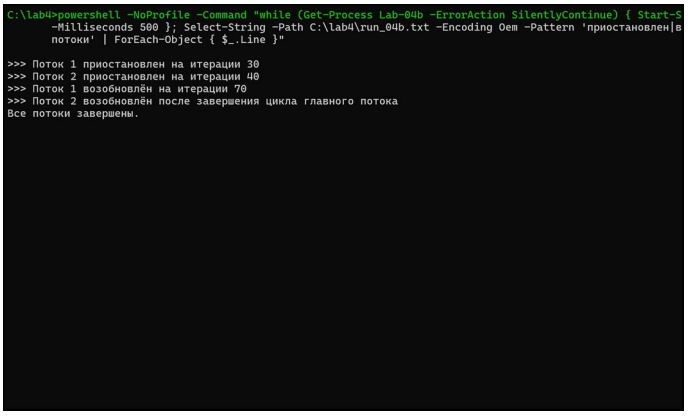



Рисунок 2.6 – *Lab*-04*b*: приостановка и возобновление потоков на заданных итерациях

Строки подтверждают приостановку и возобновление в заданных итерациях.

Состояние потоков в момент приостановки показано на рисунке 2.7: один из потоков находится в состоянии *Suspended*. Для проверки в *Process Explorer* предусмотрены отдельные кадры, приведённые на рисунках 2.8 и 2.9.

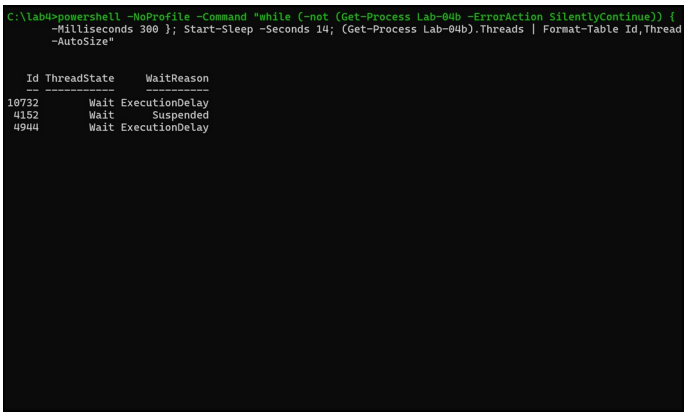



Рисунок 2.7 – *Lab*-04*b*: поток в состоянии *Suspended* по данным *PowerShell*

Первый приостановленный поток показан на рисунке 2.8, второй – на рисунке 2.9.




Рисунок 2.8 – *Process Explorer*: первый поток *Lab*-04*b* приостановлен

Второй приостановленный поток приведён на рисунке 2.9.




Рисунок 2.9 – *Process Explorer*: второй поток *Lab*-04*b* приостановлен

Оба кадра подтверждают состояние приостановленных потоков.

В приложении *Lab*-04*c* на 50-й итерации главного потока второй дочерний поток завершается принудительно функцией *TerminateThread*. Строка о завершении приведена на рисунке 2.10. Список потоков до и после завершения показан на рисунке 2.11: до завершения было три потока, после – два. Исходный текст *Lab*-04*c* приведён в листинге А.9.

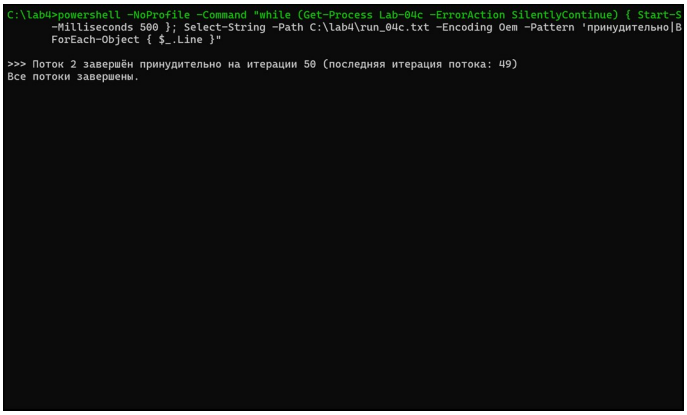



Рисунок 2.10 – *Lab*-04*c*: сообщение о принудительном завершении второго потока

Список потоков до и после завершения второго потока приведён на рисунке 2.11.

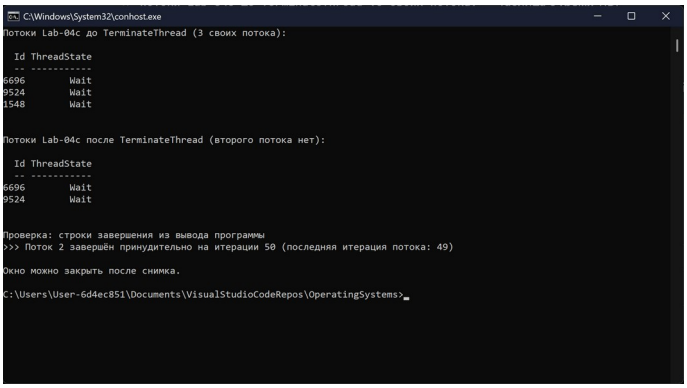



Рисунок 2.11 – *Lab*-04*c*: список потоков до и после *TerminateThread*

Список потоков сокращается с трёх до двух.

После *TerminateThread* в процессе остаются главный поток и первый дочерний поток. Отдельный кадр со списком потоков после завершения приведён на рисунке 2.12. Проверка исчезновения потока в *Process Explorer* показана на рисунках 2.13 и 2.14.

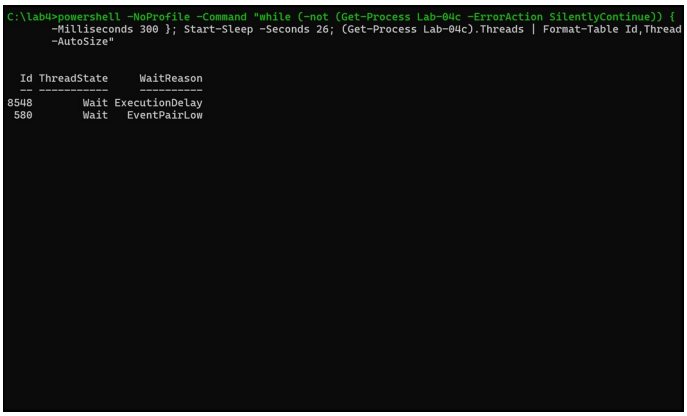



Рисунок 2.12 – *Lab*-04*c*: два потока после завершения второго потока

Список потоков до завершения показан на рисунке 2.13, после завершения – на рисунке 2.14.




Рисунок 2.13 – *Process Explorer*: три потока *Lab*-04*c* до *TerminateThread*

Состояние процесса после завершения второго потока показано на рисунке 2.14.




Рисунок 2.14 – *Process Explorer*: два потока *Lab*-04*c* после *TerminateThread*

После завершения второго потока в процессе остаются два потока.

Приложение *Lab*-04*d* перечисляет потоки собственного процесса через *Tool Help Library*. Для каждого потока выводится идентификатор и приоритет. Результат работы приведён на рисунке 2.15. Исходный текст *Lab*-04*d* приведён в листинге А.10.

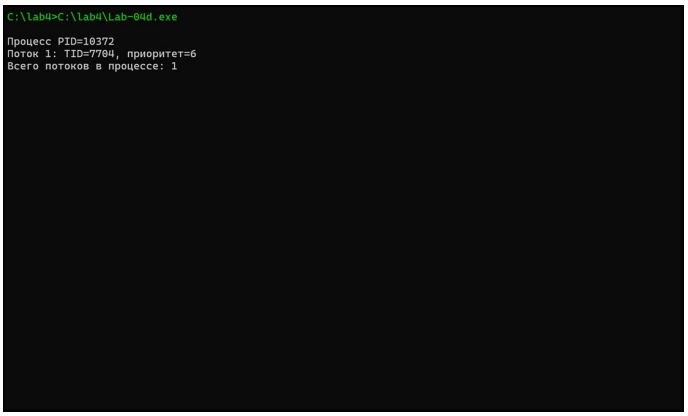



Рисунок 2.15 – *Lab*-04*d*: список потоков процесса через *Tool Help Library*

Приложение *Lab*-04*d* показывает, что в момент запуска в процессе существует только главный поток. При необходимости список может быть дополнен дочерними потоками, созданными перед вызовом *Tool Help Library*.

### 2.3 Уровень 7–8: простые числа и *TLS*

Приложение *Lab*-04*p* делит заданный диапазон на равные части и запускает заданное число потоков. Каждый поток ищет простые числа в своём диапазоне, складывает их в буфер, расположенный в *TLS* через *TlsAlloc* и *TlsSetValue*, а затем переносит результаты в общий массив.

Позиция в общем массиве определяется атомарным *InterlockedIncrement*. Синхронизация записи самих чисел не выполняется, поскольку каждый поток пишет в свою позицию. После ожидания всех потоков массив сортируется и выводится. Результат запуска *Lab*-04*p* приведён на рисунке 2.16.

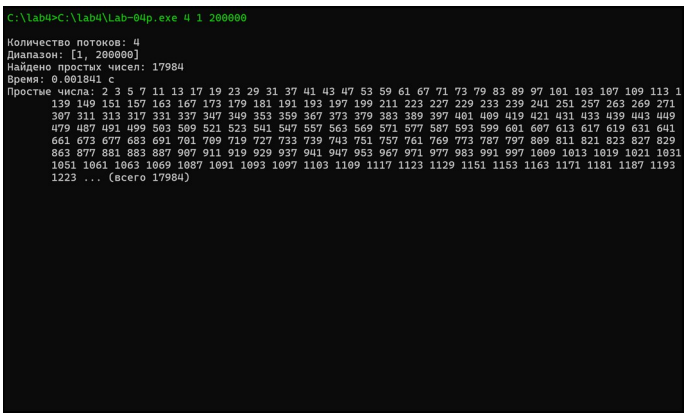



Рисунок 2.16 – *Lab*-04*p*: поиск простых чисел в четырёх потоках с *TLS* через *WinAPI*

Количество найденных чисел совпадает с табличным значением.

Приложение *Lab*-04*p*2 повторяет *Lab*-04*p*, но использует модификатор *__declspec*(*thread*). Результат показан на рисунке 2.17: количество найденных простых чисел совпадает с *Lab*-04*p*. Исходные тексты *Lab*-04*p* и *Lab*-04*p*2 приведены в листингах А.11 и А.12.

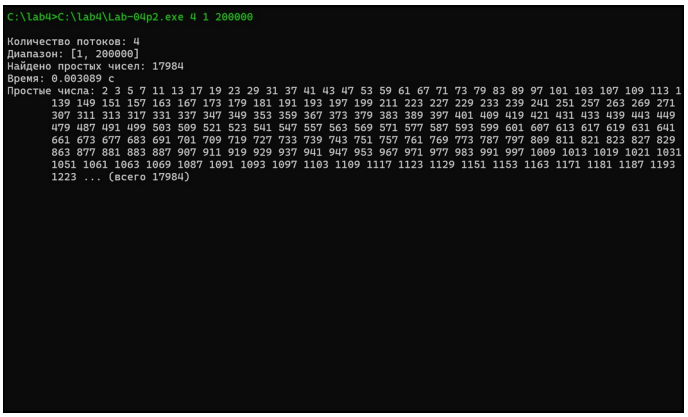



Рисунок 2.17 – *Lab*-04*p*2: тот же поиск с *TLS* через *__declspec*(*thread*)

Вся информация о простых числах сохраняется в целости. Буфер в *TLS* у каждого потока свой, поэтому потоки не перезаписывают промежуточные результаты друг друга. Общая позиция выделяется атомарно, поэтому ни одно число не теряется и не дублируется.

Серия замеров для *Windows* выполнена для 1, 2, 4, 8 и 12 потоков на диапазонах 1–1048576 и 1–2097152. Результаты приведены в таблице 2.1.

Таблица 2.1 – Время поиска простых чисел в *Windows*, секунды

| Потоков | *Lab*-04*p*, 2^20 | *Lab*-04*p*2, 2^20 | *Lab*-04*p*, 2^21 | *Lab*-04*p*2, 2^21 |
|---|---|---|---|---|
| 1 | 0,049542 | 0,036846 | 0,128653 | 0,087529 |
| 2 | 0,031881 | 0,021768 | 0,085730 | 0,056648 |
| 4 | 0,017595 | 0,014537 | 0,048412 | 0,029951 |
| 8 | 0,015689 | 0,011845 | 0,046709 | 0,037448 |
| 12 | 0,011816 | 0,013908 | 0,031437 | 0,035110 |

С ростом числа потоков время уменьшается, однако после шести потоков ускорение замедляется: физических ядер шесть, поэтому дополнительные потоки

конкурируют за те же ядра. Разница между *Lab*-04*p* и *Lab*-04*p*2 невелика и находится в пределах колебаний времени выполнения.

Сводная таблица замеров, полученная на машине, приведена на рисунке 2.18.




Рисунок 2.18 – Сводная таблица замеров *Lab*-04*p* и *Lab*-04*p*2 для *Windows*

Таблица подтверждает совпадение количества найденных простых чисел для обоих вариантов *TLS*.

### 2.4 Уровень 9–10: волокна и число Пи

Приложение *Lab*-04*pi* вычисляет число Пи по ряду Лейбница. Каждое волокно считает свою частичную сумму, после чего переключается на главное волокно. Количество волокон и число элементов на одно переключение задаются аргументами командной строки.

Работа волокон показана на рисунке 2.19. Приложение работает до нажатия *Ctrl+C*; обработчик *SetConsoleCtrlHandler* устанавливает флаг завершения, после чего главное волокно собирает частичные суммы и выводит приближение числа Пи.

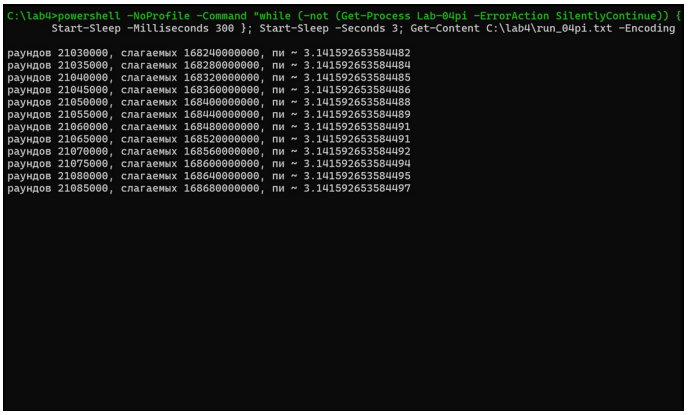



Рисунок 2.19 – *Lab*-04*pi*: работа волокон и промежуточные значения числа Пи

Результат после *Ctrl+C* приведён на рисунке 2.20. Приближение совпадает с эталонным значением до десятых долей миллиардной, а тест подтверждает корректную доставку *Ctrl+C* и код возврата 0. Исходный текст *Lab*-04*pi* приведён в листинге А.13.

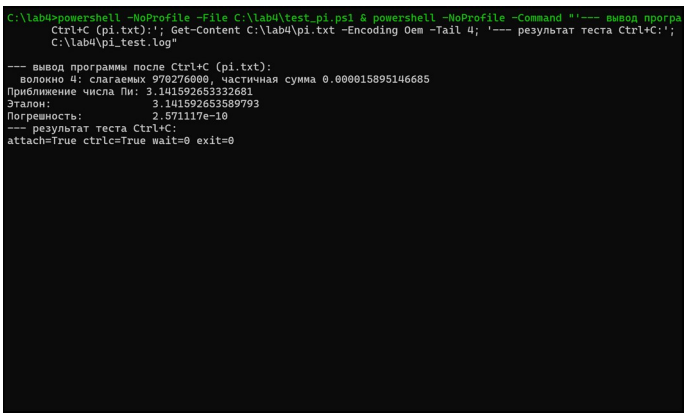



Рисунок 2.20 – *Lab*-04*pi*: приближение числа Пи и погрешность после *Ctrl+C*

Погрешность приближения не превышает $2{,}6 \cdot 10^{-10}$.

Для выбора количества элементов на переключение предусмотрена отдельная серия запусков. Её результаты приведены на рисунке 2.21; после проверки на машине они будут заменены фактическими значениями. Серия позволяет выбрать порог, при котором число обрабатываемых слагаемых максимально, а задержка реакции на *Ctrl+C* остаётся малой.



Рисунок 2.21 – *Lab*-04*pi*: подбор количества элементов на переключение

Для дальнейших запусков выбран порог, при котором переключение волокон остаётся достаточно редким, а реакция на *Ctrl+C* – быстрой.

### 2.5 Выводы по разделу

*Windows*-часть выполнена на уровнях 4–5, 7–8 и 9–10. Проверены создание потоков, приостановка и возобновление, принудительное завершение, перечисление потоков через *Tool Help Library*, *TLS* через *WinAPI* и __*declspec*(*thread*), а также волокна и обработка *Ctrl+C*.

Замеры показывают ускорение при росте числа потоков до числа физических ядер. Принудительное завершение потока освобождает его ресурсы, но не

позволяет корректно завершить выполняемую функцию, поэтому в прикладных программах его следует избегать.

# 3 Выполнение заданий для *Linux*

## 3.1 Подготовка среды и сборка

*Linux*-часть выполнена на *Arch Linux ARM* с компилятором *GCC* 16.1.1. Исходные тексты находятся в каталоге *lab*4/*linux*, сборка выполняется сценарием *build.sh* с ключами *-O*2, *-pthread*, *-Wall* и *-Wextra*. Собираются пять приложений: *Lab*-04*px*, *Lab*-04*p*, *Lab*-04*p*2, *Lab*-04*pi* и *Lab*-03*primes*. Общие функции вынесены в листинг А.6.

Для проверки использованы утилита *ps* с ключом *-L* и файловая система /*proc*. Рабочий каталог на машине – ~/*OS*4, исполняемые файлы собраны в нём же.

## 3.2 Уровень 4–5: базовые операции с потоками

Функция *lab*04*x* повторяет логику *Windows*-версии: печатает *PID*, *TID*, номер итерации, символ пользователя и символ ФИО, после чего задерживается на 400 мс. Идентификатор потока получен системным вызовом *gettid*.

Приложение *Lab*-04*px* создаёт два дочерних потока на 50 и 125 итераций, а главный поток выполняет 100 итераций и ожидает завершения обоих через *pthread_join*. Список потоков работающего процесса показан на рисунке 3.1: видны три строки *ps -L*, строка *Threads*: 3 и три каталога в /*proc*/*PID*/*task*. Исходный текст *Lab*-04*px* приведён в листинге А.1.

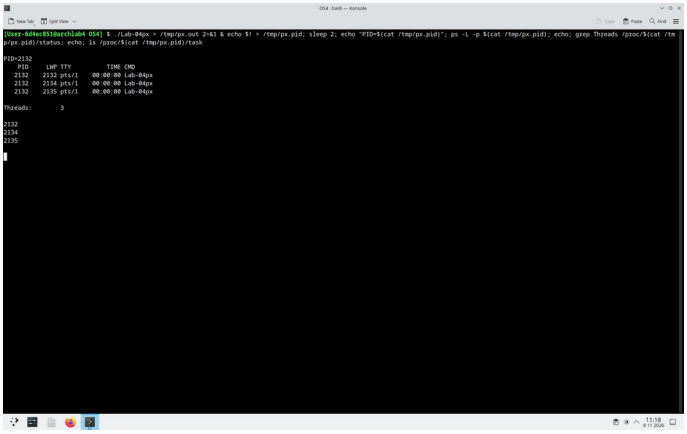



Рисунок 3.1 – *Lab*-04*px*: список потоков через *ps -L*, /*proc*/*PID*/*status* и /*proc*/*PID*/*task*

Список потоков подтверждает наличие трёх потоков процесса.

После завершения всех потоков программа выводит сообщение и завершается с кодом 0. Проверка завершения приведена на рисунке 3.2, а фрагмент вывода *Lab*-04*px* – на рисунке 3.3.

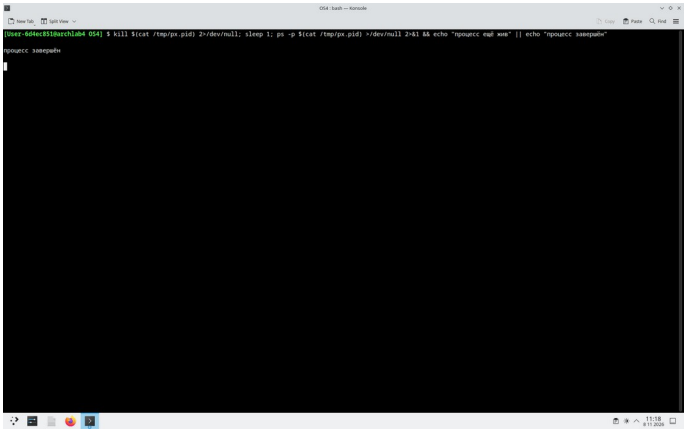


Рисунок 3.2 – *Lab*-04*px*: завершение процесса после ожидания всех потоков

Фрагмент вывода *Lab*-04*px* приведён на рисунке 3.3.

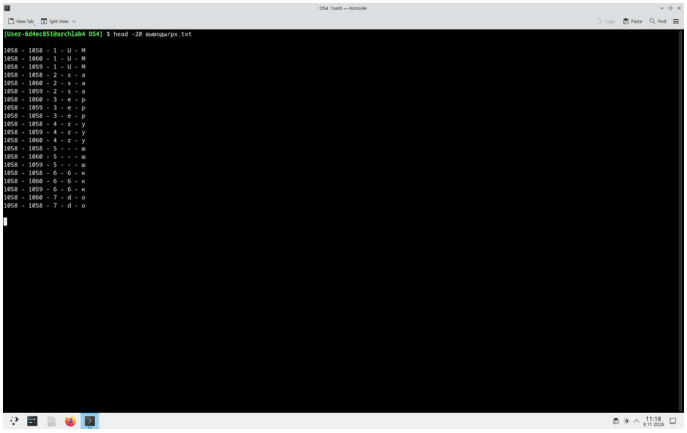


Рисунок 3.3 – *Lab*-04*px*: вывод идентификаторов и итераций трёх потоков

Вывод подтверждает, что все три потока работают с общим идентификатором процесса и разными идентификаторами потоков. Задержка между итерациями составляет 400 мс.

### 3.3 Уровень 7–8: простые числа и *TLS*

Приложение *Lab*-04*p* ищет простые числа в потоках. Диапазон делится на равные части, каждая часть обрабатывается своим потоком. Буфер найденных чисел хранится в *TLS*: ключ создаётся один раз через *pthread_once*, доступ выполняется через *pthread_setspecific* и *pthread_getspecific*.

После заполнения буфера поток переносит числа в общий массив. Позиция выделяется атомарной операцией *atomic_fetch_add*, поэтому потери и дублирования не возникают. Результат работы *Lab*-04*p* приведён на рисунке 3.4.

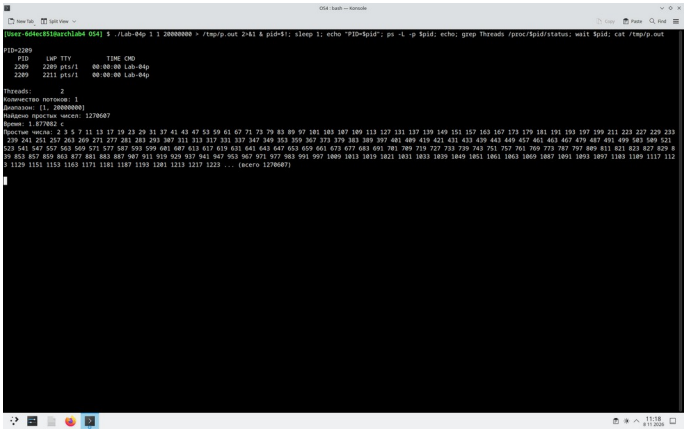


Рисунок 3.4 – *Lab*-04*p*: поиск простых чисел в потоках с *TLS* через *pthread_once*

Приложение *Lab*-04*p*2 повторяет *Lab*-04*p*, но использует модификатор *__thread*. Результат показан на рисунке 3.5: количество простых чисел совпадает. Исходные тексты *Lab*-04*p*, *Lab*-04*p*2 и *Lab*-03*primes* приведены в листингах А.2, А.3 и А.5.

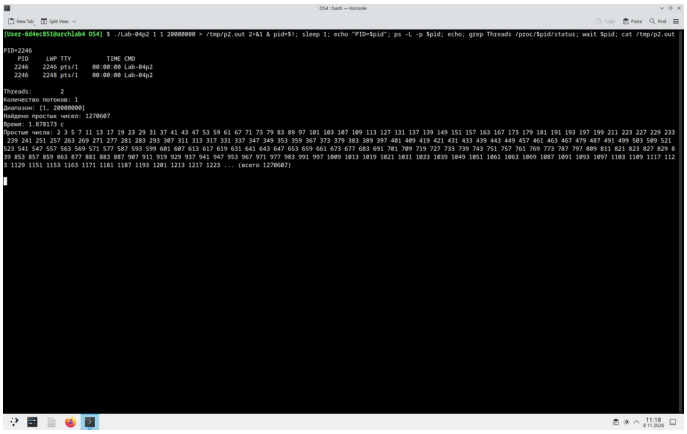


Рисунок 3.5 – *Lab*-04*p*2: тот же поиск с *TLS* через *__thread*

Серия замеров для *Linux* выполнена для 1, 2, 4, 8 и 12 потоков и процессов на диапазонах 1–1048576 и 1–2097152. В качестве процессного аналога использовано приложение *Lab*-03*primes*, которое повторяет схему ЛР3: делит диапазон между процессами и собирает результаты через каналы.

Результаты замеров приведены в таблице 3.1, а сводный вывод серии – на рисунке 3.6.

Таблица 3.1 – Время поиска простых чисел в *Linux*, секунды

| *N* | *Lab*-04*p*, 2^20 | *Lab*-04*p*2, 2^20 | *Lab*-03*primes*, 2^20 | *Lab*-04*p*, 2^21 | *Lab*-04*p*2, 2^21 | *Lab*-03*primes*, 2^21 |
|---|---|---|---|---|---|---|
| 1 | 0,039961 | 0,034111 | 0,070123 | 0,088762 | 0,117350 | 0,151877 |
| 2 | 0,020081 | 0,019990 | 0,050556 | 0,051696 | 0,052594 | 0,138710 |
| 4 | 0,011063 | 0,011210 | 0,026886 | 0,030031 | 0,028529 | 0,103032 |
| 8 | 0,008189 | 0,008383 | 0,019264 | 0,020299 | 0,021342 | 0,039416 |
| 12 | 0,007446 | 0,009383 | 0,018662 | 0,018582 | 0,017135 | 0,039192 |

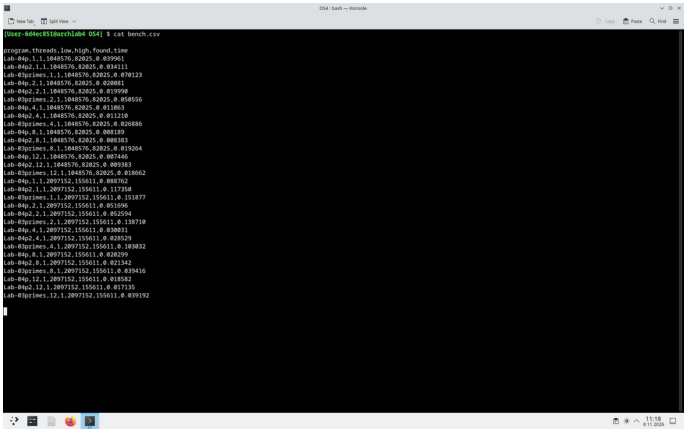



Рисунок 3.6 – Сводная таблица замеров *Lab*-04*p*, *Lab*-04*p*2 и *Lab*-03*primes*

Потоки обгоняют процессы на всех конфигурациях. Накладные расходы *fork*, создание каналов и чтение результатов заметно больше стоимости *pthread_create* и *pthread_join*. После шести потоков ускорение замедляется, поскольку физических ядер шесть.

### 3.4 Уровень 9–10: волокна и число Пи

Приложение *Lab*-04*pi* считает число Пи по ряду Лейбница на волокнах *ucontext*. Каждое волокно имеет собственный стек, получает свою последовательность слагаемых и после заданного числа элементов переключается на главный контекст. Завершение выполняется по *SIGINT*: обработчик устанавливает флаг, после чего главный контекст собирает частичные суммы.

На рисунке 3.7 показано, что *ps -L* видит только один поток, хотя внутри работают четыре волокна. Это подтверждает, что волокна не являются потоками ядра. Исходный текст *Lab*-04*pi* приведён в листинге А.4.

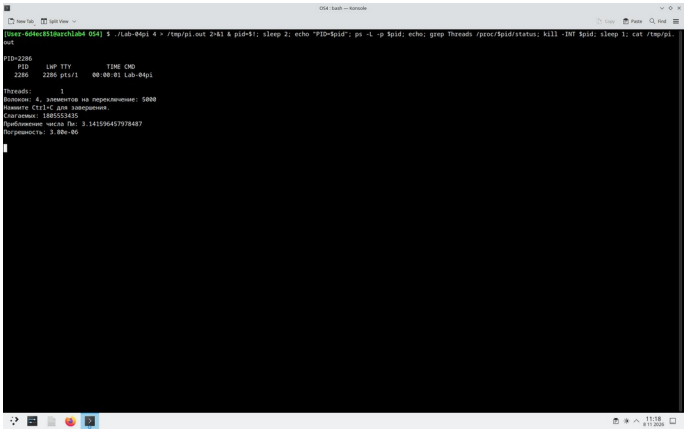



Рисунок 3.7 – *Lab*-04*pi*: один поток по данным *ps -L* при четырёх работающих волокнах и вывод числа Пи после *Ctrl+C*

Количество элементов на переключение подбиралось опытным путём. Результаты серии приведены в таблице 3.2: с ростом порога число обработанных слагаемых растёт, но после 5000–10000 элементов выигрыш становится небольшим.

Таблица 3.2 – Подбор количества элементов на переключение волокон в *Linux*

| Элементов на переключение | Слагаемых за 2 с |
|---|---|
| 500 | 924684500 |
| 1000 | 1286531000 |
| 2000 | 1578476000 |
| 5000 | 1826483749 |
| 10000 | 1901210000 |
| 20000 | 1962031855 |
| 50000 | 1998961839 |
| 100000 | 2033842216 |

Для дальнейших запусков выбран порог 5000 элементов: он входит в рекомендованный методическим указанием диапазон 1000–5000 и даёт около 1,8 млрд слагаемых за две секунды, сохраняя быструю реакцию на *Ctrl+C*.

### 3.5 Сравнение потоков и процессов

Сравнение показывает, что потоки создаются и завершаются быстрее процессов, а обмен результатами через общую память дешевле обмена через каналы. На 12 потоках время *Lab*-04*p* для диапазона 2^21 составило 0,018582 с, а для 12 процессов *Lab*-03*primes* – 0,039192 с.

Одновременно потоки требуют синхронизации доступа к общим данным. В *Lab*-04*p* и *Lab*-04*p*2 запись в общий массив защищена только атомарным выделением позиции: каждый поток пишет в свою ячейку, поэтому дополнительная блокировка не нужна.

### 3.6 Выводы по разделу

*Linux*-часть выполнена на уровнях 4–5, 7–8 и 9–10. Проверены *pthread_create* и *pthread_join*, *TLS* через *pthread_once* и *__thread*, атомарная запись в общий массив, волокна *ucontext* и обработка *SIGINT*.

Замеры подтверждают преимущество потоков перед процессами по накладным расходам. Волокна дают кооперативную многозадачность внутри одного потока и не видны ядру, что подтверждено выводом *ps -L*.

## Заключение

В ходе работы изучены потоки *Windows* и *Linux*, механизм *Thread Local Storage* и волокна. Реализованы приложения уровней 4–5, 7–8 и 9–10 для обеих операционных систем.

Проверено, что потоки разделяют адресное пространство, но имеют собственные стеки и идентификаторы. *TLS* даёт каждому потоку отдельный экземпляр буфера. Волокна переключаются кооперативно и не отображаются в списке потоков ядра.

Замеры показали ускорение при увеличении числа потоков до числа физических ядер. При дальнейшем росте числа потоков выигрыш замедляется. Принудительное завершение потока освобождает его ресурсы, но не позволяет корректно завершить работу функции, поэтому его следует применять только в крайних случаях.

# ПРИЛОЖЕНИЕ А

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#include "lab04_common.h"

#define CHILD1_ITERS 50
#define CHILD2_ITERS 125
#define MAIN_ITERS 100

static void *child1(void *arg)
{
    (void)arg;
    lab04x(CHILD1_ITERS);
    return NULL;
}

static void *child2(void *arg)
{
    (void)arg;
    lab04x(CHILD2_ITERS);
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;
    int rc;

    rc = pthread_create(&t1, NULL, child1, NULL);
    if (rc != 0)
    {
        fprintf(stderr, "Ошибка pthread_create 1: %d\n", rc);
        return 1;
    }

    rc = pthread_create(&t2, NULL, child2, NULL);
    if (rc != 0)
    {
        fprintf(stderr, "Ошибка pthread_create 2: %d\n", rc);
        pthread_join(t1, NULL);
        return 1;
    }

    lab04x(MAIN_ITERS);

    rc = pthread_join(t1, NULL);
    if (rc != 0)
    {
        fprintf(stderr, "Ошибка pthread_join 1: %d\n", rc);
        return 1;
    }
```

```
    rc = pthread_join(t2, NULL);
    if (rc != 0)
    {
        fprintf(stderr, "Ошибка pthread_join 2: %d\n", rc);
        return 1;
    }

    printf("Все потоки завершены.\n");
    return 0;
}
```

Листинг А.1 – *Lab*-04*px.c* (*Linux*)

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_PRIMES 4000000

typedef struct
{
    int low;
    int high;
} RANGE;

static pthread_once_t g_once = PTHREAD_ONCE_INIT;
static pthread_key_t g_key;
static int *g_all;
static atomic_int g_count;

static void make_key(void)
{
    pthread_key_create(&g_key, free);
}

static int is_prime(int n)
{
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
    {
        if (n % d == 0) return 0;
    }
    return 1;
}

static void *prime_worker(void *arg)
{
    RANGE *r = (RANGE *)arg;
    int cap = r->high - r->low + 1;
    if (cap < 1) cap = 1;

    pthread_once(&g_once, make_key);

    int *buf = (int *)malloc((size_t)cap * sizeof(int));
    if (buf == NULL)
    {
        return NULL;
    }
    pthread_setspecific(g_key, buf);

    int n = 0;
```

```
    for (int v = r->low; v <= r->high; v++)
    {
        if (is_prime(v))
        {
            buf[n++] = v;
        }
    }

    int *local = (int *)pthread_getspecific(g_key);
    for (int i = 0; i < n; i++)
    {
        int pos = atomic_fetch_add(&g_count, 1);
        if (pos < MAX_PRIMES)
        {
            g_all[pos] = local[i];
        }
    }

    return NULL;
}

static int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[])
{
    if (argc != 4)
    {
        fprintf(stderr, "Использование: %s <Количество потоков>
<Нижний порог> <Верхний порог>\n", argv[0]);
        return 1;
    }

    int threads = atoi(argv[1]);
    int low = atoi(argv[2]);
    int high = atoi(argv[3]);

    if (threads < 1 || low < 1 || low > high)
    {
        fprintf(stderr, "Ошибка: требуется потоков >= 1, 1 <= L <=
R\n");
        return 1;
    }

    g_all = (int *)malloc((size_t)MAX_PRIMES * sizeof(int));
    if (g_all == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        return 1;
```

```
    }

    atomic_store(&g_count, 0);

    pthread_t *tids = (pthread_t *)malloc((size_t)threads *
sizeof(pthread_t));
    RANGE *ranges = (RANGE *)malloc((size_t)threads * sizeof(RANGE));
    if (tids == NULL || ranges == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        free(tids);
        free(ranges);
        free(g_all);
        return 1;
    }

    int total = high - low + 1;
    int chunk = total / threads;
    int rest = total % threads;
    int start = low;

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    for (int i = 0; i < threads; i++)
    {
        int part = chunk + (i < rest ? 1 : 0);
        int finish = start + part - 1;
        if (i == threads - 1) finish = high;

        ranges[i].low = start;
        ranges[i].high = finish;

        int rc = pthread_create(&tids[i], NULL, prime_worker,
&ranges[i]);
        if (rc != 0)
        {
            fprintf(stderr, "Ошибка pthread_create %d: %d\n", i + 1,
rc);
            for (int k = 0; k < i; k++)
            {
                pthread_join(tids[k], NULL);
            }
            free(tids);
            free(ranges);
            free(g_all);
            return 1;
        }
        start = finish + 1;
    }

    for (int i = 0; i < threads; i++)
    {
```

```
        pthread_join(tids[i], NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);

    int found = atomic_load(&g_count);
    if (found > MAX_PRIMES) found = MAX_PRIMES;

    qsort(g_all, (size_t)found, sizeof(int), cmp_int);

    double elapsed = (double)(t1.tv_sec - t0.tv_sec) + (double)
(t1.tv_nsec - t0.tv_nsec) / 1e9;

    printf("Количество потоков: %d\n", threads);
    printf("Диапазон: [%d, %d]\n", low, high);
    printf("Найдено простых чисел: %d\n", found);
    printf("Время: %.6f c\n", elapsed);
    printf("Простые числа:");
    for (int i = 0; i < found && i < 200; i++)
    {
        printf(" %d", g_all[i]);
    }
    if (found > 200) printf(" ... (всего %d)", found);
    printf("\n");

    free(tids);
    free(ranges);
    free(g_all);
    return 0;
}
```

Листинг А.2 – *Lab*-04*p.c* (*Linux*)

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_PRIMES 4000000

typedef struct
{
    int low;
    int high;
} RANGE;

static __thread int *g_buf;
static int *g_all;
static atomic_int g_count;

static int is_prime(int n)
{
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
    {
        if (n % d == 0) return 0;
    }
    return 1;
}

static void *prime_worker(void *arg)
{
    RANGE *r = (RANGE *)arg;
    int cap = r->high - r->low + 1;
    if (cap < 1) cap = 1;

    g_buf = (int *)malloc((size_t)cap * sizeof(int));
    if (g_buf == NULL)
    {
        return NULL;
    }

    int n = 0;
    for (int v = r->low; v <= r->high; v++)
    {
        if (is_prime(v))
        {
            g_buf[n++] = v;
        }
    }

    for (int i = 0; i < n; i++)
```

```
    {
        int pos = atomic_fetch_add(&g_count, 1);
        if (pos < MAX_PRIMES)
        {
            g_all[pos] = g_buf[i];
        }
    }

    free(g_buf);
    g_buf = NULL;
    return NULL;
}

static int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[])
{
    if (argc != 4)
    {
        fprintf(stderr, "Использование: %s <Количество потоков>
<Нижний порог> <Верхний порог>\n", argv[0]);
        return 1;
    }

    int threads = atoi(argv[1]);
    int low = atoi(argv[2]);
    int high = atoi(argv[3]);

    if (threads < 1 || low < 1 || low > high)
    {
        fprintf(stderr, "Ошибка: требуется потоков >= 1, 1 <= L <=
R\n");
        return 1;
    }

    g_all = (int *)malloc((size_t)MAX_PRIMES * sizeof(int));
    if (g_all == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        return 1;
    }

    atomic_store(&g_count, 0);

    pthread_t *tids = (pthread_t *)malloc((size_t)threads *
sizeof(pthread_t));
    RANGE *ranges = (RANGE *)malloc((size_t)threads * sizeof(RANGE));
    if (tids == NULL || ranges == NULL)
```

```
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        free(tids);
        free(ranges);
        free(g_all);
        return 1;
    }

    int total = high - low + 1;
    int chunk = total / threads;
    int rest = total % threads;
    int start = low;

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    for (int i = 0; i < threads; i++)
    {
        int part = chunk + (i < rest ? 1 : 0);
        int finish = start + part - 1;
        if (i == threads - 1) finish = high;

        ranges[i].low = start;
        ranges[i].high = finish;

        int rc = pthread_create(&tids[i], NULL, prime_worker,
&ranges[i]);
        if (rc != 0)
        {
            fprintf(stderr, "Ошибка pthread_create %d: %d\n", i + 1,
rc);
            for (int k = 0; k < i; k++)
            {
                pthread_join(tids[k], NULL);
            }
            free(tids);
            free(ranges);
            free(g_all);
            return 1;
        }
        start = finish + 1;
    }

    for (int i = 0; i < threads; i++)
    {
        pthread_join(tids[i], NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);

    int found = atomic_load(&g_count);
    if (found > MAX_PRIMES) found = MAX_PRIMES;
```

```
    qsort(g_all, (size_t)found, sizeof(int), cmp_int);

    double elapsed = (double)(t1.tv_sec - t0.tv_sec) + (double)
(t1.tv_nsec - t0.tv_nsec) / 1e9;

    printf("Количество потоков: %d\n", threads);
    printf("Диапазон: [%d, %d]\n", low, high);
    printf("Найдено простых чисел: %d\n", found);
    printf("Время: %.6f c\n", elapsed);
    printf("Простые числа:");
    for (int i = 0; i < found && i < 200; i++)
    {
        printf(" %d", g_all[i]);
    }
    if (found > 200) printf(" ... (всего %d)", found);
    printf("\n");

    free(tids);
    free(ranges);
    free(g_all);
    return 0;
}
```

Листинг А.3 – *Lab*-04*p*2.*c* (*Linux*)

```
#define _GNU_SOURCE
#include <math.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ucontext.h>

#define STACK_SIZE (256 * 1024)

static volatile sig_atomic_t g_stop = 0;
static int g_fibers = 0;
static long g_chunk = 5000;
static double *g_sums = NULL;
static unsigned long long *g_terms = NULL;
static int *g_done = NULL;
static ucontext_t g_main_ctx;
static ucontext_t *g_ctx = NULL;
static char **g_stacks = NULL;

static void on_sigint(int sig)
{
    (void)sig;
    g_stop = 1;
}

static void fiber_main(int id)
{
    unsigned long long i = (unsigned long long)id;
    unsigned long long count = 0;
    double sum = 0.0;

    while (!g_stop)
    {
        for (long k = 0; k < g_chunk && !g_stop; k++)
        {
            double term = 1.0 / (2.0 * (double)i + 1.0);
            if (i & 1ULL)
            {
                sum -= term;
            }
            else
            {
                sum += term;
            }
            i += (unsigned long long)g_fibers;
            count++;
        }
        g_sums[id] = sum;
        g_terms[id] = count;
        swapcontext(&g_ctx[id], &g_main_ctx);
    }
```

```
    g_sums[id] = sum;
    g_terms[id] = count;
    g_done[id] = 1;
    swapcontext(&g_ctx[id], &g_main_ctx);

    for (;;)
    {
    }
}

int main(int argc, char *argv[])
{
    if (argc < 2 || argc > 3)
    {
        fprintf(stderr, "Использование: %s <Количество волокон>
[элементов на переключение]\n", argv[0]);
        return 1;
    }

    g_fibers = atoi(argv[1]);
    if (g_fibers < 1)
    {
        fprintf(stderr, "Ошибка: количество волокон должно быть >=
1\n");
        return 1;
    }

    if (argc == 3)
    {
        g_chunk = atol(argv[2]);
        if (g_chunk < 1)
        {
            fprintf(stderr, "Ошибка: количество элементов должно быть
>= 1\n");
            return 1;
        }
    }

    g_sums = (double *)calloc((size_t)g_fibers, sizeof(double));
    g_terms = (unsigned long long *)calloc((size_t)g_fibers,
sizeof(unsigned long long));
    g_done = (int *)calloc((size_t)g_fibers, sizeof(int));
    g_ctx = (ucontext_t *)calloc((size_t)g_fibers,
sizeof(ucontext_t));
    g_stacks = (char **)calloc((size_t)g_fibers, sizeof(char *));

    if (g_sums == NULL || g_terms == NULL || g_done == NULL || g_ctx
== NULL || g_stacks == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        free(g_sums);
        free(g_terms);
        free(g_done);
```

```
        free(g_ctx);
        free(g_stacks);
        return 1;
    }

    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = on_sigint;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) != 0)
    {
        perror("sigaction");
        free(g_sums);
        free(g_terms);
        free(g_done);
        free(g_ctx);
        free(g_stacks);
        return 1;
    }

    for (int i = 0; i < g_fibers; i++)
    {
        g_stacks[i] = (char *)malloc(STACK_SIZE);
        if (g_stacks[i] == NULL)
        {
            fprintf(stderr, "Ошибка: не хватило памяти под стек\n");
            for (int k = 0; k < i; k++) free(g_stacks[k]);
            free(g_sums);
            free(g_terms);
            free(g_done);
            free(g_ctx);
            free(g_stacks);
            return 1;
        }

        if (getcontext(&g_ctx[i]) != 0)
        {
            perror("getcontext");
            return 1;
        }

        g_ctx[i].uc_stack.ss_sp = g_stacks[i];
        g_ctx[i].uc_stack.ss_size = STACK_SIZE;
        g_ctx[i].uc_link = &g_main_ctx;

        makecontext(&g_ctx[i], (void (*)(void))fiber_main, 1, i);
    }

    printf("Волокон: %d, элементов на переключение: %ld\n", g_fibers,
g_chunk);
    printf("Нажмите Ctrl+C для завершения.\n");
    fflush(stdout);
```

```
    int running = g_fibers;
    while (running > 0)
    {
        for (int i = 0; i < g_fibers; i++)
        {
            if (!g_done[i])
            {
                swapcontext(&g_main_ctx, &g_ctx[i]);
                if (g_done[i]) running--;
            }
        }
    }

    double total = 0.0;
    unsigned long long terms = 0;

    for (int i = 0; i < g_fibers; i++)
    {
        total += g_sums[i];
        terms += g_terms[i];
    }

    double pi = 4.0 * total;
    double err = fabs(pi - 3.14159265358979323846);

    printf("Слагаемых: %llu\n", terms);
    printf("Приближение числа Пи: %.15f\n", pi);
    printf("Погрешность: %.2e\n", err);

    for (int i = 0; i < g_fibers; i++) free(g_stacks[i]);
    free(g_sums);
    free(g_terms);
    free(g_done);
    free(g_ctx);
    free(g_stacks);
    return 0;
}
```

Листинг А.4 – *Lab*-04*pi.c* (*Linux*)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define MAX_PRIMES 4000000

typedef struct
{
    int low;
    int high;
} RANGE;

static int is_prime(int n)
{
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
    {
        if (n % d == 0) return 0;
    }
    return 1;
}

static void child_run(int fd, RANGE r)
{
    for (int v = r.low; v <= r.high; v++)
    {
        if (is_prime(v))
        {
            int x = v;
            if (write(fd, &x, sizeof(x)) != (ssize_t)sizeof(x))
            {
                _exit(1);
            }
        }
    }
    _exit(0);
}

static int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[])
{
    if (argc != 4)
    {
        fprintf(stderr, "Использование: %s <Количество процессов>
```

```
<Нижний порог> <Верхний порог>\n", argv[0]);
        return 1;
    }

    int procs = atoi(argv[1]);
    int low = atoi(argv[2]);
    int high = atoi(argv[3]);

    if (procs < 1 || low < 1 || low > high)
    {
        fprintf(stderr, "Ошибка: требуется процессов >= 1, 1 <= L <=
R\n");
        return 1;
    }

    int *all = (int *)malloc((size_t)MAX_PRIMES * sizeof(int));
    int (*pipes)[2] = (int (*)[2])malloc((size_t)procs *
sizeof(int[2]));
    RANGE *ranges = (RANGE *)malloc((size_t)procs * sizeof(RANGE));
    pid_t *pids = (pid_t *)malloc((size_t)procs * sizeof(pid_t));

    if (all == NULL || pipes == NULL || ranges == NULL || pids ==
NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        free(all);
        free(pipes);
        free(ranges);
        free(pids);
        return 1;
    }

    int total = high - low + 1;
    int chunk = total / procs;
    int rest = total % procs;
    int start = low;

    for (int i = 0; i < procs; i++)
    {
        int part = chunk + (i < rest ? 1 : 0);
        int finish = start + part - 1;
        if (i == procs - 1) finish = high;

        ranges[i].low = start;
        ranges[i].high = finish;
        start = finish + 1;

        if (pipe(pipes[i]) != 0)
        {
            perror("pipe");
            for (int k = 0; k < i; k++)
            {
                close(pipes[k][0]);
```

```
                close(pipes[k][1]);
            }
            free(all);
            free(pipes);
            free(ranges);
            free(pids);
            return 1;
        }
    }

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    for (int i = 0; i < procs; i++)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork");
            for (int k = 0; k < procs; k++)
            {
                close(pipes[k][0]);
                close(pipes[k][1]);
            }
            free(all);
            free(pipes);
            free(ranges);
            free(pids);
            return 1;
        }

        if (pid == 0)
        {
            for (int k = 0; k < procs; k++)
            {
                close(pipes[k][0]);
                if (k != i) close(pipes[k][1]);
            }
            child_run(pipes[i][1], ranges[i]);
        }

        pids[i] = pid;
    }

    for (int i = 0; i < procs; i++)
    {
        close(pipes[i][1]);
    }

    int found = 0;
    for (int i = 0; i < procs; i++)
    {
        int v;
```

```
        while (read(pipes[i][0], &v, sizeof(v)) ==
(ssize_t)sizeof(v))
        {
            if (found < MAX_PRIMES)
            {
                all[found++] = v;
            }
        }
        close(pipes[i][0]);
    }

    for (int i = 0; i < procs; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);

    qsort(all, (size_t)found, sizeof(int), cmp_int);

    double elapsed = (double)(t1.tv_sec - t0.tv_sec) + (double)
(t1.tv_nsec - t0.tv_nsec) / 1e9;

    printf("Количество процессов: %d\n", procs);
    printf("Диапазон: [%d, %d]\n", low, high);
    printf("Найдено простых чисел: %d\n", found);
    printf("Время: %.6f c\n", elapsed);
    printf("Простые числа:");
    for (int i = 0; i < found && i < 200; i++)
    {
        printf(" %d", all[i]);
    }
    if (found > 200) printf(" ... (всего %d)", found);
    printf("\n");

    free(all);
    free(pipes);
    free(ranges);
    free(pids);
    return 0;
}
```

Листинг А.5 – *Lab*-03*primes.c* (*Linux*)

```
#ifndef LAB04_COMMON_H
#define LAB04_COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>

#define ITER_DELAY_US 400000
#define USER_NAME "User-6d4ec851"

static const char *FIO_CHARS[] = {
    "М", "а", "р", "у", "ш", "к", "о",
    "Т", "и", "м", "о", "ф", "е", "й"
};
#define FIO_LEN ((int)(sizeof(FIO_CHARS) / sizeof(FIO_CHARS[0])))
#define USER_LEN ((int)(sizeof(USER_NAME) - 1))

static void lab04x(int iterations)
{
    pid_t pid = getpid();
    pid_t tid = (pid_t)syscall(SYS_gettid);

    for (int i = 1; i <= iterations; i++)
    {
        char user_sym = USER_NAME[(i - 1) % USER_LEN];
        const char *fio_sym = FIO_CHARS[(i - 1) % FIO_LEN];

        printf("%d - %d - %d - %c - %s\n", (int)pid, (int)tid, i,
user_sym, fio_sym);
        fflush(stdout);
        usleep(ITER_DELAY_US);
    }
}

#endif
```

Листинг А.6 – *lab*04*_common.h* (*Linux*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"
#include "lab04_common.h"

typedef struct
{
    int iterations;
} THREAD_ARGS;

static DWORD WINAPI worker(LPVOID param)
{
    THREAD_ARGS *a = (THREAD_ARGS *)param;
    lab04x(a->iterations);
    return 0;
}

int main(void)
{
    lab04_init();

    THREAD_ARGS a1 = {55};
    THREAD_ARGS a2 = {130};

    HANDLE h[2];
    DWORD id[2];

    h[0] = CreateThread(NULL, 0, worker, &a1, 0, &id[0]);
    if (h[0] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 1: %lu\n",
GetLastError());
        lab04_done();
        return 1;
    }

    h[1] = CreateThread(NULL, 0, worker, &a2, 0, &id[1]);
    if (h[1] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 2: %lu\n",
GetLastError());
        CloseHandle(h[0]);
        lab04_done();
        return 1;
    }

    printf("Поток 1: TID=%lu, итераций 55\n", (unsigned long)id[0]);
    printf("Поток 2: TID=%lu, итераций 130\n", (unsigned long)id[1]);
    printf("Главный поток: итераций 100\n\n");
    fflush(stdout);
```

```
    lab04x(100);

    WaitForMultipleObjects(2, h, TRUE, INFINITE);
    CloseHandle(h[0]);
    CloseHandle(h[1]);

    printf("\nВсе потоки завершены.\n");
    lab04_done();
    return 0;
}
```

Листинг А.7 – *Lab*-04*a.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"
#include "lab04_common.h"

typedef struct
{
    int iterations;
} THREAD_ARGS;

static DWORD WINAPI worker(LPVOID param)
{
    THREAD_ARGS *a = (THREAD_ARGS *)param;
    lab04x(a->iterations);
    return 0;
}

int main(void)
{
    lab04_init();

    THREAD_ARGS a1 = {55};
    THREAD_ARGS a2 = {130};

    HANDLE h[2];
    DWORD id[2];

    h[0] = CreateThread(NULL, 0, worker, &a1, 0, &id[0]);
    if (h[0] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 1: %lu\n",
GetLastError());
        lab04_done();
        return 1;
    }

    h[1] = CreateThread(NULL, 0, worker, &a2, 0, &id[1]);
    if (h[1] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 2: %lu\n",
GetLastError());
        CloseHandle(h[0]);
        lab04_done();
        return 1;
    }

    printf("Поток 1: TID=%lu, итераций 55\n", (unsigned long)id[0]);
    printf("Поток 2: TID=%lu, итераций 130\n", (unsigned long)id[1]);
    printf("Главный поток: итераций 100\n\n");
    fflush(stdout);
```

```
    DWORD main_tid = GetCurrentThreadId();
    DWORD pid = GetCurrentProcessId();

    for (int i = 1; i <= 100; i++)
    {
        char user_sym = USER_NAME[(i - 1) % USER_LEN];
        const char *fio_sym = FIO_CHARS[(i - 1) % FIO_LEN];

        printf("%lu - %lu - %d - %c - %s\n", (unsigned long)pid,
(unsigned long)main_tid, i, user_sym, fio_sym);
        fflush(stdout);

        if (i == 30)
        {
            SuspendThread(h[0]);
            printf("\n>>> Поток 1 приостановлен на итерации %d\n\n",
i);
            fflush(stdout);
        }
        if (i == 70)
        {
            ResumeThread(h[0]);
            printf("\n>>> Поток 1 возобновлён на итерации %d\n\n",
i);
            fflush(stdout);
        }
        if (i == 40)
        {
            SuspendThread(h[1]);
            printf("\n>>> Поток 2 приостановлен на итерации %d\n\n",
i);
            fflush(stdout);
        }

        Sleep(ITER_DELAY_MS);
    }

    ResumeThread(h[1]);
    printf("\n>>> Поток 2 возобновлён после завершения цикла главного
потока\n\n");
    fflush(stdout);

    WaitForMultipleObjects(2, h, TRUE, INFINITE);
    CloseHandle(h[0]);
    CloseHandle(h[1]);

    printf("\nВсе потоки завершены.\n");
    lab04_done();
    return 0;
}
```

Листинг А.8 – *Lab*-04*b.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"
#include "lab04_common.h"

typedef struct
{
    int iterations;
} THREAD_ARGS;

static DWORD WINAPI worker(LPVOID param)
{
    THREAD_ARGS *a = (THREAD_ARGS *)param;
    lab04x(a->iterations);
    return 0;
}

int main(void)
{
    lab04_init();

    THREAD_ARGS a1 = {55};
    THREAD_ARGS a2 = {130};

    HANDLE h[2];
    DWORD id[2];

    h[0] = CreateThread(NULL, 0, worker, &a1, 0, &id[0]);
    if (h[0] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 1: %lu\n",
GetLastError());
        lab04_done();
        return 1;
    }

    h[1] = CreateThread(NULL, 0, worker, &a2, 0, &id[1]);
    if (h[1] == NULL)
    {
        fprintf(stderr, "Ошибка CreateThread 2: %lu\n",
GetLastError());
        CloseHandle(h[0]);
        lab04_done();
        return 1;
    }

    printf("Поток 1: TID=%lu, итераций 55\n", (unsigned long)id[0]);
    printf("Поток 2: TID=%lu, итераций 130\n", (unsigned long)id[1]);
    printf("Главный поток: итераций 100\n\n");
    fflush(stdout);
```

```
    DWORD main_tid = GetCurrentThreadId();
    DWORD pid = GetCurrentProcessId();

    for (int i = 1; i <= 100; i++)
    {
        char user_sym = USER_NAME[(i - 1) % USER_LEN];
        const char *fio_sym = FIO_CHARS[(i - 1) % FIO_LEN];

        printf("%lu - %lu - %d - %c - %s\n", (unsigned long)pid,
(unsigned long)main_tid, i, user_sym, fio_sym);
        fflush(stdout);

        if (i == 50)
        {
            DWORD code = 0;
            if (TerminateThread(h[1], 0))
            {
                printf("\n>>> Поток 2 завершён принудительно на
итерации %d (последняя итерация потока: %d)\n\n",
                       i, lab04x_iteration(id[1]));
                fflush(stdout);
            }
            else
            {
                fprintf(stderr, "Ошибка TerminateThread: %lu\n",
GetLastError());
            }
            (void)code;
        }

        Sleep(ITER_DELAY_MS);
    }

    WaitForSingleObject(h[0], INFINITE);
    WaitForSingleObject(h[1], INFINITE);

    printf("\nДескрипторы закрываются: поток 1 (TID=%lu), поток 2
(TID=%lu)\n",
           (unsigned long)id[0], (unsigned long)id[1]);
    CloseHandle(h[0]);
    CloseHandle(h[1]);

    printf("Все потоки завершены.\n");
    lab04_done();
    return 0;
}
```

Листинг А.9 – *Lab*-04*c.c* (*Windows*)

```
#include <windows.h>
#include <tlhelp32.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"

int main(void)
{
    DWORD pid = GetCurrentProcessId();

    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    if (snap == INVALID_HANDLE_VALUE)
    {
        fprintf(stderr, "Ошибка CreateToolhelp32Snapshot: %lu\n",
GetLastError());
        return 1;
    }

    THREADENTRY32 te;
    te.dwSize = sizeof(THREADENTRY32);

    printf("Процесс PID=%lu\n", (unsigned long)pid);

    if (!Thread32First(snap, &te))
    {
        fprintf(stderr, "Ошибка Thread32First: %lu\n",
GetLastError());
        CloseHandle(snap);
        return 1;
    }

    int count = 0;
    do
    {
        if (te.th32OwnerProcessID == pid)
        {
            count++;
            printf("Поток %d: TID=%lu, приоритет=%ld\n", count,
(unsigned long)te.th32ThreadID, (long)te.tpBasePri);
        }
        te.dwSize = sizeof(THREADENTRY32);
    } while (Thread32Next(snap, &te));

    CloseHandle(snap);

    printf("Всего потоков в процессе: %d\n", count);
    return 0;
}
```

Листинг А.10 – *Lab*-04*d.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"

#define MAX_PRIMES 4000000

typedef struct
{
    int low;
    int high;
    int limit;
} RANGE;

static DWORD g_tls;
static int *g_all;
static volatile LONG g_count;

static int is_prime(int n)
{
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
    {
        if (n % d == 0) return 0;
    }
    return 1;
}

static DWORD WINAPI prime_worker(LPVOID param)
{
    RANGE *r = (RANGE *)param;
    int cap = r->high - r->low + 1;
    if (cap < 1) cap = 1;

    int *buf = (int *)HeapAlloc(GetProcessHeap(), 0, (SIZE_T)cap *
sizeof(int));
    if (buf == NULL)
    {
        return 1;
    }
    TlsSetValue(g_tls, buf);

    int n = 0;
    for (int v = r->low; v <= r->high && n < r->limit; v++)
    {
        if (is_prime(v))
        {
            buf[n++] = v;
        }
    }
```

```
    int *local = (int *)TlsGetValue(g_tls);
    for (int i = 0; i < n; i++)
    {
        LONG pos = InterlockedIncrement(&g_count) - 1;
        if (pos < MAX_PRIMES)
        {
            g_all[pos] = local[i];
        }
    }

    return 0;
}

static int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[])
{
    if (argc != 4)
    {
        fprintf(stderr, "Использование: %s <Количество потоков>
<Нижний порог> <Верхний порог>\n", argv[0]);
        return 1;
    }

    int threads = atoi(argv[1]);
    int low = atoi(argv[2]);
    int high = atoi(argv[3]);

    if (threads < 1 || low < 1 || low > high)
    {
        fprintf(stderr, "Ошибка: требуется потоков >= 1, 1 <= L <=
R\n");
        return 1;
    }

    g_all = (int *)malloc((SIZE_T)MAX_PRIMES * sizeof(int));
    if (g_all == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        return 1;
    }

    g_tls = TlsAlloc();
    if (g_tls == TLS_OUT_OF_INDEXES)
    {
        fprintf(stderr, "Ошибка TlsAlloc: %lu\n", GetLastError());
        free(g_all);
        return 1;
```

```
    }

    HANDLE *handles = (HANDLE *)malloc((SIZE_T)threads *
sizeof(HANDLE));
    RANGE *ranges = (RANGE *)malloc((SIZE_T)threads * sizeof(RANGE));
    if (handles == NULL || ranges == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        TlsFree(g_tls);
        free(g_all);
        return 1;
    }

    int total = high - low + 1;
    int chunk = total / threads;
    int rest = total % threads;
    int start = low;

    LARGE_INTEGER f, t0, t1;
    QueryPerformanceFrequency(&f);
    QueryPerformanceCounter(&t0);

    for (int i = 0; i < threads; i++)
    {
        int part = chunk + (i < rest ? 1 : 0);
        int finish = start + part - 1;
        if (i == threads - 1) finish = high;

        ranges[i].low = start;
        ranges[i].high = finish;
        ranges[i].limit = part;

        handles[i] = CreateThread(NULL, 0, prime_worker, &ranges[i],
0, NULL);
        if (handles[i] == NULL)
        {
            fprintf(stderr, "Ошибка CreateThread %d: %lu\n", i + 1,
GetLastError());
            for (int k = 0; k < i; k++) CloseHandle(handles[k]);
            TlsFree(g_tls);
            free(g_all);
            free(handles);
            free(ranges);
            return 1;
        }
        start = finish + 1;
    }

    WaitForMultipleObjects((DWORD)threads, handles, TRUE, INFINITE);
    for (int i = 0; i < threads; i++) CloseHandle(handles[i]);

    QueryPerformanceCounter(&t1);
```

```
    int found = (int)g_count;
    if (found > MAX_PRIMES) found = MAX_PRIMES;

    qsort(g_all, (size_t)found, sizeof(int), cmp_int);

    printf("Количество потоков: %d\n", threads);
    printf("Диапазон: [%d, %d]\n", low, high);
    printf("Найдено простых чисел: %d\n", found);
    printf("Время: %.6f c\n", (double)(t1.QuadPart - t0.QuadPart) /
(double)f.QuadPart);
    printf("Простые числа:");
    for (int i = 0; i < found && i < 200; i++)
    {
        printf(" %d", g_all[i]);
    }
    if (found > 200) printf(" ... (всего %d)", found);
    printf("\n");

    TlsFree(g_tls);
    free(g_all);
    free(handles);
    free(ranges);
    return 0;
}
```

Листинг А.11 – *Lab*-04*p.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

#include "console_ru.h"

#define MAX_PRIMES 4000000

typedef struct
{
    int low;
    int high;
    int limit;
} RANGE;

static int *g_all;
static volatile LONG g_count;

static __declspec(thread) int tls_count;
static __declspec(thread) int tls_buf[200000];

static int is_prime(int n)
{
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    for (int d = 3; (long long)d * d <= n; d += 2)
    {
        if (n % d == 0) return 0;
    }
    return 1;
}

static DWORD WINAPI prime_worker(LPVOID param)
{
    RANGE *r = (RANGE *)param;

    tls_count = 0;
    for (int v = r->low; v <= r->high && tls_count < r->limit &&
tls_count < 200000; v++)
    {
        if (is_prime(v))
        {
            tls_buf[tls_count++] = v;
        }
    }

    for (int i = 0; i < tls_count; i++)
    {
        LONG pos = InterlockedIncrement(&g_count) - 1;
        if (pos < MAX_PRIMES)
        {
            g_all[pos] = tls_buf[i];
        }
```

```
    }

    return 0;
}

static int cmp_int(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);
}

int main(int argc, char *argv[])
{
    if (argc != 4)
    {
        fprintf(stderr, "Использование: %s <Количество потоков>
<Нижний порог> <Верхний порог>\n", argv[0]);
        return 1;
    }

    int threads = atoi(argv[1]);
    int low = atoi(argv[2]);
    int high = atoi(argv[3]);

    if (threads < 1 || low < 1 || low > high)
    {
        fprintf(stderr, "Ошибка: требуется потоков >= 1, 1 <= L <=
R\n");
        return 1;
    }

    g_all = (int *)malloc((SIZE_T)MAX_PRIMES * sizeof(int));
    if (g_all == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        return 1;
    }

    HANDLE *handles = (HANDLE *)malloc((SIZE_T)threads *
sizeof(HANDLE));
    RANGE *ranges = (RANGE *)malloc((SIZE_T)threads * sizeof(RANGE));
    if (handles == NULL || ranges == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        free(g_all);
        return 1;
    }

    int total = high - low + 1;
    int chunk = total / threads;
    int rest = total % threads;
    int start = low;
```

```
    LARGE_INTEGER f, t0, t1;
    QueryPerformanceFrequency(&f);
    QueryPerformanceCounter(&t0);

    for (int i = 0; i < threads; i++)
    {
        int part = chunk + (i < rest ? 1 : 0);
        int finish = start + part - 1;
        if (i == threads - 1) finish = high;

        ranges[i].low = start;
        ranges[i].high = finish;
        ranges[i].limit = part;

        handles[i] = CreateThread(NULL, 0, prime_worker, &ranges[i],
0, NULL);
        if (handles[i] == NULL)
        {
            fprintf(stderr, "Ошибка CreateThread %d: %lu\n", i + 1,
GetLastError());
            for (int k = 0; k < i; k++) CloseHandle(handles[k]);
            free(g_all);
            free(handles);
            free(ranges);
            return 1;
        }
        start = finish + 1;
    }

    WaitForMultipleObjects((DWORD)threads, handles, TRUE, INFINITE);
    for (int i = 0; i < threads; i++) CloseHandle(handles[i]);

    QueryPerformanceCounter(&t1);

    int found = (int)g_count;
    if (found > MAX_PRIMES) found = MAX_PRIMES;

    qsort(g_all, (size_t)found, sizeof(int), cmp_int);

    printf("Количество потоков: %d\n", threads);
    printf("Диапазон: [%d, %d]\n", low, high);
    printf("Найдено простых чисел: %d\n", found);
    printf("Время: %.6f c\n", (double)(t1.QuadPart - t0.QuadPart) /
(double)f.QuadPart);
    printf("Простые числа:");
    for (int i = 0; i < found && i < 200; i++)
    {
        printf(" %d", g_all[i]);
    }
    if (found > 200) printf(" ... (всего %d)", found);
    printf("\n");
```

```
    free(g_all);
    free(handles);
    free(ranges);
    return 0;
}
```

Листинг А.12 – *Lab*-04*p*2.*c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#include "console_ru.h"

#define CHUNK_DEFAULT 2000

typedef struct
{
    LPVOID main_fiber;
    long long start;
    long long done;
    double sum;
    int id;
} FIBER_CTX;

static volatile LONG g_stop = 0;
static long long g_terms = 0;
static int g_chunk = CHUNK_DEFAULT;
static int g_fibers = 1;

static BOOL WINAPI ctrl_handler(DWORD type)
{
    if (type == CTRL_C_EVENT || type == CTRL_BREAK_EVENT || type ==
CTRL_CLOSE_EVENT)
    {
        InterlockedExchange(&g_stop, 1);
        return TRUE;
    }
    return FALSE;
}

static VOID CALLBACK fiber_proc(LPVOID param)
{
    FIBER_CTX *ctx = (FIBER_CTX *)param;
    int chunk = g_chunk;

    for (;;)
    {
        for (int i = 0; i < chunk; i++)
        {
            long long k = ctx->start + i;
            double term = 1.0 / (double)(2 * k + 1);
            if (k & 1LL)
            {
                ctx->sum -= term;
            }
            else
            {
                ctx->sum += term;
            }
```

```
        }
        ctx->start += (long long)chunk * g_fibers;
        ctx->done += chunk;
        InterlockedExchangeAdd64(&g_terms, chunk);
        SwitchToFiber(ctx->main_fiber);
    }
}

int main(int argc, char *argv[])
{
    int fibers_count = 4;
    int chunk = CHUNK_DEFAULT;

    if (argc >= 2)
    {
        fibers_count = atoi(argv[1]);
    }
    if (argc >= 3)
    {
        chunk = atoi(argv[2]);
    }
    if (fibers_count < 1 || chunk < 1)
    {
        fprintf(stderr, "Использование: %s <Количество волокон>
<Элементов на переключение>\n", argv[0]);
        return 1;
    }

    g_fibers = fibers_count;

    if (!SetConsoleCtrlHandler(ctrl_handler, TRUE))
    {
        fprintf(stderr, "Ошибка SetConsoleCtrlHandler: %lu\n",
GetLastError());
        return 1;
    }

    LPVOID main_fiber = ConvertThreadToFiber(NULL);
    if (main_fiber == NULL)
    {
        fprintf(stderr, "Ошибка ConvertThreadToFiber: %lu\n",
GetLastError());
        return 1;
    }

    FIBER_CTX *ctx = (FIBER_CTX *)calloc((size_t)fibers_count,
sizeof(FIBER_CTX));
    LPVOID *workers = (LPVOID *)calloc((size_t)fibers_count,
sizeof(LPVOID));
    if (ctx == NULL || workers == NULL)
    {
        fprintf(stderr, "Ошибка: не хватило памяти\n");
        ConvertFiberToThread();
```

```
        return 1;
    }

    for (int i = 0; i < fibers_count; i++)
    {
        ctx[i].main_fiber = main_fiber;
        ctx[i].start = (long long)i * chunk;
        ctx[i].sum = 0.0;
        ctx[i].id = i;

        workers[i] = CreateFiber(0, fiber_proc, &ctx[i]);
        if (workers[i] == NULL)
        {
            fprintf(stderr, "Ошибка CreateFiber %d: %lu\n", i + 1,
GetLastError());
            for (int k = 0; k < i; k++) DeleteFiber(workers[k]);
            ConvertFiberToThread();
            free(ctx);
            free(workers);
            return 1;
        }
    }

    printf("Волокон: %d, элементов на переключение: %d\n",
fibers_count, chunk);
    printf("Нажмите Ctrl+C для завершения и вывода приближения числа
Пи\n\n");
    fflush(stdout);

    long long rounds = 0;
    while (g_stop == 0)
    {
        for (int i = 0; i < fibers_count && g_stop == 0; i++)
        {
            SwitchToFiber(workers[i]);
        }

        rounds++;
        if ((rounds % 5000) == 0)
        {
            double sum = 0.0;
            for (int i = 0; i < fibers_count; i++) sum += ctx[i].sum;
            printf("раундов %lld, слагаемых %lld, пи ~ %.15f\n",
rounds, g_terms, 4.0 * sum);
            fflush(stdout);
        }
    }

    double total = 0.0;
    for (int i = 0; i < fibers_count; i++)
    {
        total += ctx[i].sum;
    }
```

```
    printf("\nОбработано слагаемых: %lld\n", g_terms);
    for (int i = 0; i < fibers_count; i++)
    {
        printf("  волокно %d: слагаемых %lld, частичная сумма
%.15f\n", i + 1, ctx[i].done, ctx[i].sum);
    }
    printf("Приближение числа Пи: %.15f\n", 4.0 * total);
    printf("Эталон:               %.15f\n", 3.141592653589793);
    printf("Погрешность:          %.6e\n", fabs(4.0 * total -
3.141592653589793));

    for (int i = 0; i < fibers_count; i++)
    {
        DeleteFiber(workers[i]);
    }
    ConvertFiberToThread();
    free(ctx);
    free(workers);

    return 0;
}
```

Листинг А.13 – *Lab*-04*pi.c* (*Windows*)

```
#ifndef LAB04_COMMON_H
#define LAB04_COMMON_H

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define ITER_DELAY_MS 400
#define USER_NAME "User-6d4ec851"

static const char *FIO_CHARS[] = {
    "\xd0\x9c", "\xd0\xb0", "\xd1\x80", "\xd1\x83", "\xd1\x88",
"\xd0\xba", "\xd0\xbe",
    "\xd0\xa2", "\xd0\xb8", "\xd0\xbc", "\xd0\xbe", "\xd1\x84",
"\xd0\xb5", "\xd0\xb9"
};
#define FIO_LEN ((int)(sizeof(FIO_CHARS) / sizeof(FIO_CHARS[0])))
#define USER_LEN ((int)(sizeof(USER_NAME) - 1))

static int g_current_iter[256];
static CRITICAL_SECTION g_iter_lock;

static void lab04_init(void)
{
    InitializeCriticalSection(&g_iter_lock);
    memset(g_current_iter, 0, sizeof(g_current_iter));
}

static void lab04_done(void)
{
    DeleteCriticalSection(&g_iter_lock);
}

static void lab04x(int iterations)
{
    DWORD pid = GetCurrentProcessId();
    DWORD tid = GetCurrentThreadId();

    for (int i = 1; i <= iterations; i++)
    {
        char user_sym = USER_NAME[(i - 1) % USER_LEN];
        const char *fio_sym = FIO_CHARS[(i - 1) % FIO_LEN];

        printf("%lu - %lu - %d - %c - %s\n", (unsigned long)pid,
(unsigned long)tid, i, user_sym, fio_sym);
        fflush(stdout);

        EnterCriticalSection(&g_iter_lock);
        g_current_iter[tid & 0xFF] = i;
        LeaveCriticalSection(&g_iter_lock);
```

```
        Sleep(ITER_DELAY_MS);
    }
}

static int lab04x_iteration(DWORD tid)
{
    int v;
    EnterCriticalSection(&g_iter_lock);
    v = g_current_iter[tid & 0xFF];
    LeaveCriticalSection(&g_iter_lock);
    return v;
}

#endif
```

Листинг А.14 – *lab*04*_common.h* (*Windows*)

```
#ifndef CONSOLE_RU_H
#define CONSOLE_RU_H

#include <windows.h>
#include <stdio.h>
#include <stdarg.h>

static const char *utf8_to_oem(const char *src, char *out, int cap)
{
    int wlen = MultiByteToWideChar(CP_UTF8, 0, src, -1, NULL, 0);
    if (wlen <= 0 || wlen > 4096) {
        return src;
    }
    wchar_t wbuf[4096];
    MultiByteToWideChar(CP_UTF8, 0, src, -1, wbuf, wlen);
    int olen = WideCharToMultiByte(866, 0, wbuf, -1, out, cap, NULL,
NULL);
    if (olen <= 0) {
        return src;
    }
    return out;
}

static int rprintf(const char *fmt, ...)
{
    char tmp[4096];
    char conv[4096];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(tmp, sizeof(tmp), fmt, ap);
    va_end(ap);
    fputs(utf8_to_oem(tmp, conv, sizeof(conv)), stdout);
    fflush(stdout);
    return 0;
}

static int rfprintf(FILE *stream, const char *fmt, ...)
{
    char tmp[4096];
    char conv[4096];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(tmp, sizeof(tmp), fmt, ap);
    va_end(ap);
    fputs(utf8_to_oem(tmp, conv, sizeof(conv)), stream);
    fflush(stream);
    return 0;
}

#define printf  rprintf
#define fprintf rfprintf
```

```
#endif
```

Листинг А.15 – *console_ru.h* (*Windows*)